



Go from Data to Strategy: Tepper School of Business

# How to Implement Tool Calling with Gemma 4 and Python

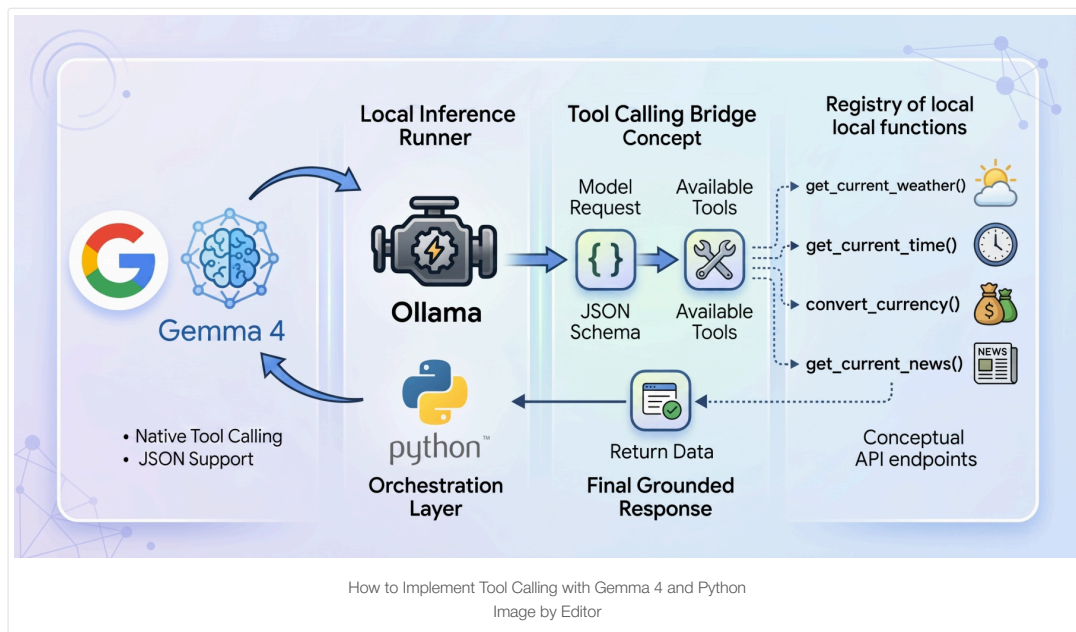
by **Matthew Mayo** on April 14, 2026 in **Artificial Intelligence** 0



In this article, you will learn how to build a local, privacy-first tool-calling agent using the Gemma 4 model family and Ollama.

Topics we will cover include:

- An overview of the Gemma 4 model family and its capabilities.
- How tool calling enables language models to interact with external functions.
- How to implement a local tool calling system using Python and Ollama.



## Introducing the Gemma 4 Family

The open-weights model ecosystem shifted recently with the release of the **Gemma 4 model family**. Built by Google, the Gemma 4 variants were created with the intention of providing frontier-level capabilities under a permissive Apache 2.0 license, enabling machine learning practitioners complete control over their infrastructure and data privacy.

The Gemma 4 release features models ranging from the parameter-dense 31B and structurally complex 26B Mixture of Experts (MoE) to lightweight, edge-focused variants. More importantly for AI engineers, the model family features native support for agentic workflows. They have been fine-tuned to reliably generate structured JSON outputs and natively invoke function calls based on system instructions. This transforms them from “fingers crossed” reasoning engines into practical systems capable of executing workflows and conversing with external APIs locally.

## Tool Calling in Language Models

Language models began life as closed-loop conversationalists. If you asked a language model for real-world sensor reading or live market rates, it could at best apologize, and at worst, hallucinate an answer. Tool calling, aka function calling, is the foundational architecture shift required to fix this gap.

Tool calling serves as the bridge that can help transform static models into dynamic autonomous agents. When tool calling is enabled, the model evaluates a user prompt against a provided registry of available programmatic tools (supplied via JSON schema). Rather than attempting to guess the answer using only internal weights, the model pauses inference, formats a structured request specifically designed to trigger an external function, and awaits the result. Once the result is processed by the host application and handed back to the model, the model synthesizes the injected live context to formulate a grounded final response.

## The Setup: Ollama and Gemma 4:E2B

To build a genuinely local, private-first tool calling system, we will use **Ollama** as our local inference runner, paired with the `gemma4:e2b` (Edge 2 billion parameter) model.

The `gemma4:e2b` model is built specifically for mobile devices and IoT applications. It represents a paradigm shift in what is possible on consumer hardware, activating an effective 2 billion parameter footprint during inference. This optimization preserves system memory while achieving near-zero latency execution. By executing entirely offline, it removes rate limits and API costs while preserving strict data privacy.

Despite this incredibly small size, Google has engineered `gemma4:e2b` to inherit the multimodal properties and native function-calling capabilities of the larger 31B model, making it an ideal foundation for a fast, responsive desktop agent. It also allows us to test for the capabilities of the new model family without requiring a GPU.

## The Code: Setting Up the Agent

To orchestrate the language model and the tool interfaces, we will rely on a zero-dependency philosophy for our implementation, leveraging only standard Python libraries like `urllib` and `json`, ensuring maximum portability and transparency while also avoiding bloat.

The complete code for this tutorial can be found at this [GitHub repository](#).

The architectural flow of our application operates in the following way:

1. Define local Python functions that act as our tools
2. Define a strict JSON schema that explains to the language model exactly what these tools do and what parameters they expect
3. Pass the user's query and the tool registry to the local Ollama API
4. Catch the model's response, identify if it requested a tool call, execute the corresponding local code, and feed the answer back

## Building the Tools: `get_current_weather`

Let's dive into the code, keeping in mind that our agent's capability rests on the quality of its underlying functions. Our first function is `get_current_weather`, which reaches out to the open-source Open-Meteo API to resolve real-time weather data for a specific location.

```
1 def get_current_weather(city: str, unit: str = "celsius") -> str:
2     """Gets the current temperature for a given city using open-meteo API."""
3     try:
4         # Geocode the city to get latitude and longitude
5         geo_url = f"https://geocoding-api.open-meteo.com/v1/search?name={urllib.parse.quote(city)}&count=1"
6         geo_req = urllib.request.Request(geo_url, headers={'User-Agent': 'Gemma4ToolCalling/1.0'})
7         with urllib.request.urlopen(geo_req) as response:
8             geo_data = json.loads(response.read().decode('utf-8'))
9
10        if "results" not in geo_data or not geo_data["results"]:
11            return f"Could not find coordinates for city: {city}."
12
13        location = geo_data["results"][0]
14        lat = location["latitude"]
15        lon = location["longitude"]
16        country = location.get("country", "")
17
18        # Fetch the weather
19        temp_unit = "fahrenheit" if unit.lower() == "fahrenheit" else "celsius"
20        weather_url = f"https://api.open-meteo.com/v1/forecast?latitude={lat}&longitude={lon}&current=temperature_2m,wind_speed_10m&tempe"
21        weather_req = urllib.request.Request(weather_url, headers={'User-Agent': 'Gemma4ToolCalling/1.0'})
22        with urllib.request.urlopen(weather_req) as response:
23            weather_data = json.loads(response.read().decode('utf-8'))
24
25        if "current" in weather_data:
26            current = weather_data["current"]
27            temp = current["temperature_2m"]
28            wind = current["wind_speed_10m"]
29            temp_unit_str = weather_data["current_units"]["temperature_2m"]
30            wind_unit_str = weather_data["current_units"]["wind_speed_10m"]
31
32            return f"The current weather in {city.title()} ({country}) is {temp}{temp_unit_str} with wind speeds of {wind}{wind_unit_str}"
33        else:
34            return f"Weather data for {city} is unavailable from the API."
35
36    except Exception as e:
37        return f"Error fetching weather for {city}: {e}"
```

This Python function implements a two-stage API resolution pattern. Because standard weather APIs typically require strict geographical coordinates, our function transparently intercepts the city string provided by the model and geocodes it into latitude and longitude coordinates. With the coordinates formatted, it invokes the weather forecast endpoint and constructs a concise natural language string representing the telemetry point.

However, writing the function in Python is only half the execution. The model needs to be informed visually about this tool. We do this by mapping the Python function into an Ollama-compliant JSON schema dictionary:

```
1 {
2     "type": "function",
3     "function": {
4         "name": "get_current_weather",
5         "description": "Gets the current temperature for a given city.",
6         "parameters": {
```

```

7         "type": "object",
8         "properties": {
9             "city": {
10                "type": "string",
11                "description": "The city name, e.g. Tokyo"
12            },
13            "unit": {
14                "type": "string",
15                "enum": ["celsius", "fahrenheit"]
16            }
17        },
18        "required": ["city"]
19    }
20 }
21 }

```

This rigid structural blueprint is critical, as it explicitly details variable expectations, strict string enums, and required parameters, all of which guide the gemma4:e2b weights into reliably generating syntax-perfect calls.

## Tool Calling Under the Hood

The core of the autonomous workflow happens primarily inside the main loop orchestrator. Once a user issues a prompt, we establish the initial JSON payload for the Ollama API, explicitly linking gemma4:e2b and appending the global array containing our parsed toolkit.

```

1  # Initial payload to the model
2  messages = [{"role": "user", "content": user_query}]
3  payload = {
4      "model": "gemma4:e2b",
5      "messages": messages,
6      "tools": available_tools,
7      "stream": False
8  }
9
10 try:
11     response_data = call_ollama(payload)
12 except Exception as e:
13     print(f"Error calling Ollama API: {e}")
14     return
15
16 message = response_data.get("message", {})

```

Once the initial web request resolves, it is critical that we evaluate the architecture of the returned message block. We are not blindly assuming text exists here. The model, aware of the active tools, will signal its desired outcome by attaching a tool\_calls dictionary.

If tool\_calls exist, we pause the standard synthesis workflow, parse the requested function name out of the dictionary block, execute the Python tool with the parsed kwargs dynamically, and inject the returned live data back into the conversational array.

```

1  # Check if the model decided to call tools
2  if "tool_calls" in message and message["tool_calls"]:
3
4      # Add the model's tool calls to the chat history
5      messages.append(message)
6
7      # Execute each tool call
8      num_tools = len(message["tool_calls"])
9      for i, tool_call in enumerate(message["tool_calls"]):
10         function_name = tool_call["function"]["name"]
11         arguments = tool_call["function"]["arguments"]
12
13         if function_name in TOOL_FUNCTIONS:
14             func = TOOL_FUNCTIONS[function_name]
15             try:
16                 # Execute the underlying Python function
17                 result = func(**arguments)
18
19                 # Add the tool response to messages history
20                 messages.append({
21                     "role": "tool",
22                     "content": str(result),
23                     "name": function_name
24                 })
25             except TypeError as e:
26                 print(f"Error calling function: {e}")
27         else:
28             print(f"Unknown function: {function_name}")
29
30     # Send the tool results back to the model to get the final answer
31     payload["messages"] = messages
32
33     try:
34         final_response_data = call_ollama(payload)
35         print("[RESPONSE]")
36         print(final_response_data.get("message", {}).get("content", "")+"\n")
37     except Exception as e:
38         print(f"Error calling Ollama API for final response: {e}")

```

Notice the important secondary interaction: once the dynamic result is appended as a “tool” role, we bundle the messages history up a second time and trigger the API again. This second pass is what allows the gemma4:e2b reasoning engine to read the telemetry strings it previously hallucinated around, bridging the final gap to output the data logically in human terms.

## More Tools: Expanding the Tool Calling Capabilities

With the architectural foundation complete, enriching our capabilities requires nothing more than adding modular Python functions. Using the identical methodology described above, we incorporate three additional live tools:

1. `get_current_news`: Utilizing **NewsAPI** endpoints, this function parses arrays of global headlines based on queried keyword topics that the model identifies as contextually relevant
2. `get_current_time`: By referencing **TimeAPI.io**, this deterministic function bridges complex real-world timezone logic and offsets back into native, readable datetime strings
3. `convert_currency`: Relying on the live **ExchangeRate-API**, this function enables mathematical tracking and fractional conversion computations between fiat currencies

Each capability is processed through the JSON schema registry, expanding the baseline model's utility without requiring external orchestration or heavy dependencies.

## Testing the Tools

And now we test our tool calling.

Let's start with the first function we created, `get_current_weather`, with the following query:

“

*What is the weather in Ottawa?*

```
(tool_calling) matt@matt-macbook gemma_4_tool_calling % python main.py "What is the weather in Ottawa?"

[SYSTEM]
  o Tool: convert_currency..... [LOADED]
  o Tool: get_current_news..... [LOADED]
  o Tool: get_current_time..... [LOADED]
  o Tool: get_current_weather..... [LOADED]

[PROMPT]
What is the weather in Ottawa?

[EXECUTION]
  • Querying model...

[TOOL EXECUTION]
  └─ Calling: get_current_weather
    └─ Args: city='Ottawa'
      └─ Result: The current weather in Ottawa (Canada) is 7.8°C with wind speeds of 18.0km/h.

[EXECUTION]
  • Synthesizing results...

[RESPONSE]
The current weather in Ottawa (Canada) is 7.8°C with wind speeds of 18.0km/h.

(tool_calling) matt@matt-macbook gemma_4_tool_calling %
```

What is the weather in Ottawa?

You can see our CLI UI provides us with:

- confirmation of the available tools
- the user prompt
- details on tool execution, including the function used, the arguments sent, and the response
- the the language model's response

It appears as though we have a successful first run.

Next, let's try out another of our tools independently, namely `convert_currency`:

“

*Given the current currency exchange rate, how much is 1200 Canadian dollars in euros?*

```
(tool_calling) matt@matt-macbook gemma_4_tool_calling % python main.py "Given the current currency exchange rate, how much is 1200 Canadian dollars in euros?"

[SYSTEM]
  o Tool: convert_currency..... [LOADED]
  o Tool: get_current_news..... [LOADED]
  o Tool: get_current_time..... [LOADED]
  o Tool: get_current_weather..... [LOADED]

[PROMPT]
Given the current currency exchange rate, how much is 1200 Canadian dollars in euros?

[EXECUTION]
  • Querying model...

[TOOL EXECUTION]
  └─ Calling: convert_currency
    └─ Args: amount=1200, from_currency='CAD', to_currency='EUR'
      └─ Result: 1200 CAD is equal to 740.40 EUR (Rate: 0.617).

[EXECUTION]
  • Synthesizing results...

[RESPONSE]
1200 Canadian dollars is equal to 740.40 euros.
```

Given the current currency exchange rate, how much is 1200 Canadian dollars in euros?

More winning.

Now, let's stack tool calling requests. Let's also keep in mind that we are using a 4 billion parameter model that has half of its parameters active at any one time during inference:

“

*I am going to France next week. What is the current time in Paris? How many euros would 1500 Canadian dollars be? what is the current weather there? what is the latest news about Paris?*

```
(tool_calling) matt@matt-macbook gemma_4_tool_calling % python main.py "I am going to France next week. What is the current time in Paris? How many euros would 1500 Canadian dollars be? what is the current weather there? what is the latest news about Paris?"

[SYSTEM]
  o Tool: convert_currency..... [LOADED]
  o Tool: get_current_news..... [LOADED]
  o Tool: get_current_time..... [LOADED]
  o Tool: get_current_weather..... [LOADED]

[PROMPT]
I am going to France next week. What is the current time in Paris? How many euros would 1500 Canadian dollars be? what is the current weather there? what is the latest news about Paris?

[EXECUTION]
  • Querying model...

[TOOL EXECUTION]
  └─ Calling: get_current_time
    └─ Args: timezone='Europe/Paris'
      └─ Result: The current time in Europe/Paris is 21:59 on 04/11/2026.
  └─ Calling: convert_currency
    └─ Args: amount=1500, from_currency='CAD', to_currency='EUR'
      └─ Result: 1500 CAD is equal to 925.50 EUR (Rate: 0.617).
  └─ Calling: get_current_weather
    └─ Args: city='Paris'
      └─ Result: The current weather in Paris (France) is 9.5°C with wind speeds of 5.9km/h.
  └─ Calling: get_current_news
    └─ Args: topic='Paris'
      └─ Result: News on Paris:
          1. McSharry sets new Irish record in 200m breaststroke (BBC News)
          2. La carrera por el Mundial de Clubes: cómo quedaron Nacional y Peñarol en el ranking tras debutar en Libertadores (Diario EL PAIS Uruguay)
          3. Who needs a purse? Martha Stewart prefers these chic hands-free phone cases (Page Six)
          4. Team News: Four Liverpool stars set to miss Fulham clash (Anfieldindex.com)
          5. Trump lanserer plan for massiv triumfbue i Washington (NRK)

[EXECUTION]
  • Synthesizing results...

[RESPONSE]
Here is the information you requested:

* **Current Time in Paris:** The current time in Paris (Europe/Paris) is 21:59 on 04/11/2026.
* **Currency Conversion:** 1500 Canadian dollars is equal to 925.50 Euros.
* **Current Weather in Paris:** The current weather in Paris (France) is 9.5°C with wind speeds of 5.9km/h.
* **Latest News about Paris:**
  * McSharry sets new Irish record in 200m breaststroke (BBC News)
  * La carrera por el Mundial de Clubes: cómo quedaron Nacional y Peñarol en el ranking tras debutar en Libertadores (Diario EL PAIS Uruguay)
  * Who needs a purse? Martha Stewart prefers these chic hands-free phone cases (Page Six)
  * Team News: Four Liverpool stars set to miss Fulham clash (Anfieldindex.com)
  * Trump lanserer plan for massiv triumfbue i Washington (NRK)
```

I am going to France next week...

Would you look at that. All four questions answered by four different functions from the four separate tool calls. All on a local, private, incredibly small language model served by Ollama.

I ran queries on this setup over the course of the weekend, and never once did the model's reasoning fail. Never once. Hundreds of prompts. Admittedly, they were on the same four tools, but regardless of how vague my otherwise reasonable wording become, I couldn't stump it.

Gemma 4 certainly appears to be a powerhouse of a small language model reasoning engine with tool calling capabilities. I'll be turning my attention to building out a fully agentic system next, so stay tuned.

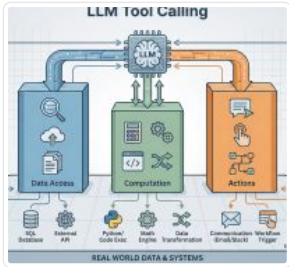
## Conclusion

The advent of tool calling behavior inside open-weight models is one of the more useful and practical developments in local AI of late. With the release of Gemma 4, we can operate securely offline, building complex systems unfettered by cloud and API restrictions. By architecturally integrating direct access to the web, local file systems, raw data processing logic, and localized APIs, even low-powered consumer devices can operate autonomously in ways that were previously restricted exclusively to cloud-tier hardware.

Share

Post

More On This Topic



Mastering LLM Tool Calling: The Complete Framework...



3 Ways of Using Gemma 2 Locally



How to Implement Resampling Methods From Scratch In Python



How To Implement Machine Learning Metrics From...



How To Implement Baseline Machine Learning...



How To Implement Simple Linear Regression From...



About Matthew Mayo

**Matthew Mayo** (@mattmayo13) holds a master's degree in computer science and a graduate diploma in data mining. As managing editor of [KDnuggets](#) & [Statology](#), and contributing editor at [Machine Learning Mastery](#), Matthew aims to make complex data science concepts accessible. His professional interests include natural language processing, language models, machine learning algorithms, and exploring emerging AI. He is driven by a mission to democratize knowledge in the data science community. Matthew has been coding since he was 6 years old.  
[View all posts by Matthew Mayo](#) →

< [Structured Outputs vs. Function Calling: Which Should Your Agent Use?](#)

[5 Techniques for Efficient Long-Context RAG](#) >

No comments yet.

Leave a Reply

Name (required)

Email (will not be published) (required)

SUBMIT COMMENT



**Welcome!**I'm *Jason Brownlee* PhDand I **help developers** get results with **machine learning**.[Read more](#)**Never miss a tutorial:****Picked for you:**[Your First Deep Learning Project in Python with Keras Step-by-Step](#)[Your First Machine Learning Project in Python Step-By-Step](#)[How to Develop LSTM Models for Time Series Forecasting](#)[How to Create an ARIMA Model for Time Series Forecasting in Python](#)[Machine Learning for Developers](#)**Loving the Tutorials?**

The [EBook Catalog](#) is where  
you'll find the **Really Good** stuff.

[>> SEE WHAT'S INSIDE](#)

Machine Learning Mastery is part of Guiding Tech Media, a leading digital media publisher focused on helping people figure out technology. [Visit our corporate website](#) to learn more about our mission and team.

[PRIVACY](#) | [DISCLAIMER](#) | [TERMS](#) | [CONTACT](#) | [SITEMAP](#) | [ADVERTISE WITH US](#)

© 2026 Guiding Tech Media All Rights Reserved